A APPENDIX

A.1 Empirical Results for HashPrune

Table Size Hash Bits	64	128	192	NoRes
8	3376.0	3545.0	3656.0	3445.0
10	3433.0	3394.0	3614.0	3445.0
12	3375.0	3559.0	3566.0	3445.0
14	3531.0	3423.0	3421.0	3445.0
16	3368.0	3653.0	3654.0	3445.0

Table 4: Average cmps for Recall > 0.95 (SPACEV-100M Dataset)

A.2 Asymptotic Analysis of PiPNN with Selected Components

Our recommended default strategies for PiPNN (according to evidence presented in Section 7) are to (1) run randomized ball carving with distributed fan-out during the partitioning phase, (2) build bidirected 2NN graphs in leaves, (3) filter candidates from the leaves via HashPrune, and (4) finish by pruning the remaining candidates using RobustPrune. We now consider the expected running time to build a PiPNN index given these choices.

First, we point out that although in some adversarial input sets certain leader choices may lead to locally-unbalanced partitioning across subproblems, it is reasonable in practice to assume that randomized ball carving divides points across subproblems roughly evenly in expectation. Under this assumption, recursive ball carving with ℓ random leaders in each subproblem terminates within $O(\log_\ell n)$ levels with high probability, where n is the number of input points. Within each level, up to fn instances of points each undergo d-dimensional distance computations with the ℓ leaders in its subproblem (where f is the amount of fanout performed at the prior level of recursion). It follows that partitioning takes $O(dfn\ell \log_\ell(n))$ time with high probability.

We stop partitioning when every leaf contains at most c points. We subsequently spend $O(|\mathcal{B}_i|^2 d)$ time for each leaf $\mathcal{B}_i$ building a $|\mathcal{B}_i| \times |\mathcal{B}_i|$ distance matrix and adding edges between each point in $\mathcal{B}_i$ and the two points from which it has smallest distance. Thus, we can charge each point in $\mathcal{B}_i$ for up to $O(|\mathcal{B}_i|d) \leq O(cd)$ time. In total, there are $\leq fn$ instances of points across all leaves, so leaf-building in total takes $O(cdfn)$ time.

HashPrune maintains a reservoir of up to $O(m)$ candidate neighbors for each point, where m is the target maximum degree of the final graph. Recall from the leaf-building step that each of the $\leq fn$ instances of points adds a candidate edge to and from each of its two nearest in-leaf neighbors. Thus, at most $4fn$ candidate neighbors are passed into HashPrune across all points. We hash each one via LSH against roughly 12 random vectors using 12 $O(d)$-time distance computations. We subsequently compare its distance from the target point with that of the furthest candidate in the reservoir (via brute force in $O(m)$ time) and the candidate already in its hash bucket if one exists (via binary search in $O(\log m)$ time). Thus, HashPrune consumes $O(fnm)$ time.

Finally, having used HashPrune to limit the number of candidates to $O(m)$ per point, RobustPrune takes only $O(m^2)$ time for each point. Thus, the final prune phase takes $O(nm^2)$ time.

Combining all components of the algorithm produces a total expected time complexity of $O(n(df\ell \log_\ell(n) + cdf + fm + m^2))$.

A.3 Proof of Determinism of PiPNN

The determinism of PiPNN follows from two propositions. Firstly, the partitioning produced by Algorithm 4 is deterministic. Thus the candidates which are given to HashPrune are always the same (though the order that they are merged into candidate lists by way of HashPrune is non-deterministic). Secondly, HashPrune itself is history independent, and so the order that points has no bearing on the final adjacency lists for each point. Thus PiPNN is deterministic. We prove the each proposition in this section.

Lemma A.1. *Randomized Ball Carving is deterministic In each subproblem the leaders are sampled randomly (Line 3), thus once randomness is fixed the same leaders will always be sampled within a given subproblem. The next subproblems are given by computing an exact nearest neighbor computation on the leaders, and thus are completely determined by the leaders and subproblem points. Merging likewise can be done in such a way as to not depend on the scheduler.*

Lemma A.2. *HashPrune is History Independent*

Let $p \subseteq \mathcal{X}$ be an arbitrary point, and consider p's HashPrune reservoir $R_p = [r_1, \ldots, r_s]$ with hash family $H = \{h_1, \ldots, h_b\}$. Suppose some fixed collection $C = \{c_1, \ldots, c_i, \ldots, c_q\}$ is to be inserted into R_p. We claim that R_p will contain the s elements of C nearest to p such that they do not collide under H. To see that this is true, consider that if we removed the size restriction of the reservoir entirely, then each position in R_p would just contain the nearest point in each hash bucket (lines 3-5 of HashPrune). Now, when we consider only the s-sized reservoir, the further member of R_p is evicted in line 10 whenever the reservoir is full. Thus, the sequence of insertions is irrelevant to the final structure of R_p.

A.4 Modifying MIRAGE

In our experiments, we compare against the MIRAGE nearest neighbor index. For transparency, we point out that, at the time of writing this, the native MIRAGE codebase does not currently support datasets much larger than the 10 million scale, because indices are done using signed 32-bit integers. Thus, a maximum degree of 128, for example, will lead to an integer overflow on the number of edges when the number of vertices exceeds 33.6 million. In order to let MIRAGE run on our datasets, we modify the code to use unsigned 64-bit integers instead, which could impact the performance of the implementation. With that said, we see little noticeable effect on the build and query speeds of the graphs. Moreover, this size integer is used in the implementations of all other algorithms that we tested.